

LD8-S3 Hybrid Tiled Direct and Overlap-Add FFT Realization Specification

Exact finite-stencil execution, bounded tile workspaces, deterministic crossover selection, and retained S2 oracle

mdstats development specification

2026-07-21

Contents

LD8-S3 - Hybrid tiled direct and overlap-add FFT realization	1
Purpose	1
Scientific operator	2
Source tiling	2
Public records	2
API	3
Input and plan identity	4
Direct-tile executor	4
FFT-tile executor	4
Linear convolution brick	4
Periodic overlap-add	4
Kernel-spectrum cache	5
Crossover selector	5
Target lookup and packed accumulation	5
FFT roundoff and supported-node repair	5
Final normalization	5
Resource accounting	5
Output metadata	6
Determinism and reproducibility	6
Failure semantics	6
Validation requirements	7
Recorded implementation evidence	7
Focused crossover benchmark	7
Full-trajectory field evidence	8
Stage boundary	8
External algorithmic source	8

LD8-S3 - Hybrid tiled direct and overlap-add FFT realization

Package target: mdstats 0.19.57a0

Status: implemented as an opt-in accelerator; production dispatch remains LD7 until LD8-S4

Primary module:

mdstats.plotting.density_tiled_fft

Purpose

LD8-S3 accelerates realization of the exact LD8-S1 finite-support density field. It partitions one globally aggregated packed CIC source into bounded logical-grid compute tiles and chooses one of two mathematically equivalent executors for each tile:

- bounded sparse direct finite-stencil scatter for fragmented or lightly occupied tiles;
- zero-padded three-dimensional FFT linear convolution followed by periodic overlap-add for sufficiently populated tiles.

The executor changes only the numerical organization of the calculation. It does not change the logical grid, Gaussian bandwidth, retained 10^{-8} Gaussian tail policy, CIC source, normalized stencil, support atlas, density units, or final normalization. LD8-S2 remains the canonical migration oracle.

Scientific operator

Let $m(\mathbf{n})$ be the globally aggregated periodic CIC mass field and let $g(\delta)$ be the exact normalized finite Gaussian stencil retained at

$$\varepsilon_{\text{tail}} = 10^{-8}.$$

The target mass is

$$M(\mathbf{t}) = \sum_{\delta \in \mathcal{K}_\varepsilon} ((\mathbf{t} - \delta) \bmod \mathbf{N}) g(\delta).$$

The scientific density is

$$\rho(\mathbf{t}) = \frac{M(\mathbf{t})}{\Delta V}, \quad \Delta V = \frac{|\det H|}{N_x N_y N_z}.$$

All supported target nodes are supplied by the exact S1 atlas. Nodes outside the atlas remain implicit zeros.

Source tiling

The occupied source nodes are partitioned by a compute-tile shape $\mathbf{T}$ that is independent of the 16^3 storage-block shape. The default is

`compute_tile_shape = (32, 32, 32)`

Each occupied source node belongs to exactly one tile,

$$\mathbf{q} = \lfloor \mathbf{n} / \mathbf{T} \rfloor.$$

Tiles are ordered by canonical C-order tile index. Partitioning is exact: no CIC source mass is duplicated or omitted.

Public records

```
@dataclass(frozen=True, slots=True)
class DensityHybridExecutorOptions:
    executor_mode: Literal["auto", "direct", "fft"]
    compute_tile_shape: tuple[int, int, int]
    pair_chunk_size: int
    min_fft_source_nodes: int
    direct_pair_seconds: float
    fft_work_seconds: float
    fft_fixed_seconds: float
    fft_workers: int
    cache_kernel_spectra: bool
```

```
@dataclass(frozen=True, slots=True)
class DensityHybridRealizationLimits:
    max_compute_tiles: int
    max_target_nodes: int
    max_direct_pairs: int
    max_fft_padded_nodes_per_tile: int
    max_lookup_bytes: int
    max_kernel_cache_bytes: int
    max_transient_bytes: int
    max_retained_bytes: int
```

```
@dataclass(frozen=True, slots=True)
class DensityHybridTilePlan:
    tile_index: tuple[int, int, int]
```

```

origin: tuple[int, int, int]
extent: tuple[int, int, int]
source_start: int
source_stop: int
source_node_count: int
source_fill_fraction: float
executor: Literal["direct", "fft"]
direct_pair_count: int
full_convolution_shape: tuple[int, int, int]
fft_padded_shape: tuple[int, int, int]
fft_padded_node_count: int
direct_cost_estimate_seconds: float
fft_cost_estimate_seconds: float
transient_bytes_estimate: int
cache_kernel_spectrum: bool

@dataclass(frozen=True, slots=True)
class DensityHybridRealizationPlan:
    source_field_identity: str
    stencil_identity: str
    routing_identity: str
    atlas_identity: str
    logical_grid_shape: tuple[int, int, int]
    compute_tile_shape: tuple[int, int, int]
    kernel_min_offset: tuple[int, int, int]
    kernel_shape: tuple[int, int, int]
    source_node_count: int
    target_support_node_count: int
    compute_tile_count: int
    direct_tile_count: int
    fft_tile_count: int
    exact_contribution_count: int
    direct_pair_count: int
    target_lookup_bytes: int
    kernel_dense_bytes: int
    kernel_spectrum_cache_bytes: int
    packed_field_bytes_upper: int
    predicted_peak_bytes: int
    tile_plans: tuple[DensityHybridTilePlan, ...]

```

Every record is immutable, schema-versioned, JSON serializable, and tied to exact input identities.

API

```

def plan_hybrid_tiled_realization(
    source_field: PeriodicPackedCICSourceField3D,
    stencil: PeriodicGaussianStencilSupport,
    routing: PeriodicKernelBlockRouting,
    atlas: DensitySupportAtlas,
    *,
    options: DensityHybridExecutorOptions | None = None,
    limits: DensityHybridRealizationLimits | None = None,
) -> DensityHybridRealizationPlan

def realize_density_hybrid_tiled(
    source_field: PeriodicPackedCICSourceField3D,
    stencil: PeriodicGaussianStencilSupport,
    routing: PeriodicKernelBlockRouting,
    atlas: DensitySupportAtlas,
    *,
    field_key: str,

```

```

label: str,
physical_units: str,
broadening_metric: str,
options: DensityHybridExecutorOptions | None = None,
limits: DensityHybridRealizationLimits | None = None,
approved_plan: DensityHybridRealizationPlan | None = None,
) -> PeriodicPackedBlockScalarField3D

```

Input and plan identity

Planning and realization require:

1. identical logical-grid shape across source, stencil, routing, and atlas;
2. identical storage-block shape across source, routing, and atlas;
3. exact source-field identity match with the atlas;
4. exact routing identity match with the atlas;
5. exact stencil identity match with the routing template;
6. exact content match between an approved plan and all four inputs;
7. nonempty output key, label, units, and broadening metric;
8. identical tile geometry and selector options between plan and realization.

Any mismatch raises `GraphAdapterError` before field realization.

Direct-tile executor

For a direct tile with source set S_q , the executor evaluates

$$M_q(\mathbf{t}) = \sum_{\mathbf{n} \in S_q} m(\mathbf{n})(\mathbf{t} - \mathbf{n}) \bmod \mathbf{N}$$

by bounded vectorized source/stencil chunks. The maximum temporary pair count is controlled by `pair_chunk_size`. The executor may allocate coordinate, lookup, and contribution arrays only for the current chunk. It may not allocate the complete source-node by stencil-offset array.

Target logical indices are mapped through the exact packed S1 support lookup. A contribution falling outside the atlas is a correctness error.

FFT-tile executor

Linear convolution brick

For an FFT tile, occupied source values are deposited into one dense local source brick covering the complete logical compute-tile extent; terminal tiles use their smaller valid extent. The exact signed stencil is reconstructed as one dense kernel brick. If the source extent is $\mathbf{E}$ and kernel extent is $\mathbf{K}$, the linear convolution shape is

$$\mathbf{C} = \mathbf{E} + \mathbf{K} - \mathbf{1}.$$

Each FFT axis is padded independently to

$$P_i = \text{next_fast_len}(C_i).$$

The tile computes

$$Y_q = \mathcal{F}^{-1}[\mathcal{F}(X_q) \mathcal{F}(G)],$$

using real transforms. Only the unpadded linear-convolution region is consumed.

Periodic overlap-add

The local output coordinate is shifted by the source-brick origin and the signed-kernel minimum, then folded modulo the logical-grid shape. The folded values are added into the exact packed target support. Source tiles are disjoint, so summing their output is exactly the periodic convolution of the complete source field.

This is a multidimensional overlap-add organization adapted from standard block-convolution methods; the triclinic metric is already encoded in the retained discrete kernel weights, so the FFT executor does not assume Cartesian separability.

Kernel-spectrum cache

Kernel spectra are reused within one realization by padded FFT shape. The approved plan places a hard byte cap on the set of spectra retained simultaneously. This per-realization cache is released with the executor and contains no field-specific source or target data beyond the selected exact stencil. A future cross-realization cache would additionally require the exact stencil identity, SciPy FFT implementation/version, and numeric representation in its key.

Crossover selector

For each tile, the planner estimates

$$C_{\text{direct}} = N_{\text{source},q} |\mathcal{K}_\varepsilon| c_{\text{pair}},$$

and

$$C_{\text{FFT}} = c_0 + c_{\text{FFT}} N_{P,q} \log_2 N_{P,q}.$$

In auto mode, FFT is selected only when:

- the source count is at least `min_fft_source_nodes`;
- the padded tile is within every FFT resource limit;
- the predicted FFT cost is lower than the predicted direct cost.

`direct` and `fft` modes force one executor for validation and benchmarking. A forced FFT tile that violates the declared limits fails transactionally rather than silently changing method.

The current cost coefficients are explicit calibration parameters. LD8-S4 must recalibrate or confirm them on the production host before default migration.

Target lookup and packed accumulation

The executor builds a sorted array of exact supported global logical indices and their packed-output positions. This lookup is bounded by `max_lookup_bytes`; it is not a dense logical-grid map.

Each tile accumulates directly into one preallocated packed mass vector. No dense global field and no list of completed dense tiles is retained. The final packed field reuses the atlas block order, occupancy bitsets, and block offsets.

FFT roundoff and supported-node repair

A true positive target value near the finite-support boundary may round to zero or a tiny negative number after FFT inversion. Supported nodes must remain strictly positive under the packed-field contract.

After all tiles are accumulated, any nonpositive supported node is recomputed directly from the exact global packed source and exact stencil. This repair:

- changes only nodes violating positivity;
- uses the canonical finite operator;
- is deterministic;
- records the repair count in metadata;
- fails if the exact recomputation is not strictly positive.

A large or systematically growing repair count is an acceptance failure in LD8-S4, not a reason to alter the scientific support.

Final normalization

The complete packed mass vector is normalized once to the requested source measure. The final floating residual is applied to the largest positive mass. Densities are obtained by division by ΔV . Per-block extrema are computed for the packed scientific field.

Resource accounting

Planning records:

- tile count and source range per tile;
- direct and FFT tile counts;
- exact source/stencil contribution count;
- direct-tile pair count;
- FFT convolution and padded shapes;
- target-lookup bytes;
- dense-kernel bytes;
- byte-bounded kernel-spectrum cache allowance;
- packed-field upper bound;
- largest mutually exclusive tile workspace;
- predicted retained-plus-maximum-transient peak.

Peak memory is modeled as

$$B_{\text{peak}} = B_{\text{retained}} + \max_q B_{\text{transient},q},$$

not as a sum of mutually exclusive tile workspaces.

Output metadata

Every realized field records at least:

```
reference_path = ld8_s3_hybrid_tiled_v1
production_backend = false
source_field_identity
stencil_identity
routing_identity
atlas_identity
compute_tile_shape
compute_tile_count
direct_tile_count
fft_tile_count
exact_contribution_count
direct_pair_count
fft_kernel_transform_count
fft_nonpositive_node_repairs
selector coefficients and executor mode
predicted peak and retained bytes
NumPy, SciPy, and FFT-worker configuration
normalization and support provenance
```

production_backend remains false until LD8-S4 passes the full acceptance gate.

Determinism and reproducibility

- Source and tile order are canonical.
- Direct chunks use canonical source and stencil order.
- Packed target order is inherited from S1.
- Repeated forced-direct runs are byte-identical on the validation platform.
- FFT runs are numerically reproducible under the recorded SciPy version, worker count, and plan; exact byte identity across libraries or thread counts is not promised.
- Plan and field JSON serialization remain canonical.

Failure semantics

The executor raises a structured error before or during realization when:

- input or plan identities disagree;
- tile count, target nodes, direct pairs, FFT padded nodes, lookup bytes, cache bytes, retained bytes, or transient bytes exceed limits;
- a forced executor is infeasible;

- a contribution maps outside the exact atlas;
- support cardinality changes;
- normalization fails;
- exact repair does not restore strict positivity.

It may not silently loosen the Gaussian cutoff, coarsen the grid, alter the kernel, omit tiles, discard supported nodes, or replace an infeasible forced path.

Validation requirements

Focused tests must cover:

- forced direct versus S2;
- forced FFT versus S2;
- automatic mixed selection;
- periodic boundary crossings;
- partial terminal blocks;
- fragmented and compact source layouts;
- oxygen-heavy occupancy;
- exact target-support identity;
- measure and units;
- deterministic direct output;
- plan JSON round trip and identity rejection;
- FFT-spectrum cache reuse;
- resource preflight;
- bounded pair chunks;
- no dense global field allocation.

Acceptance tolerances are:

$$\frac{\|\rho_{\text{direct}} - \rho_{\text{S2}}\|_1}{\|\rho_{\text{S2}}\|_1} \leq 5 \times 10^{-12},$$

and

$$\frac{\|\rho_{\text{FFT}} - \rho_{\text{S2}}\|_1}{\|\rho_{\text{S2}}\|_1} \leq 5 \times 10^{-11}.$$

Recorded implementation evidence

Focused crossover benchmark

A 96^3 production-cutoff fixture with 8,409 retained stencil offsets produced:

Case	Source nodes	Direct/FFT tiles	Hybrid time	S2 time	Speedup	Relative L^1
frag-mented	64	24 / 0	0.236 s	0.661 s	2.81x	2.90×10^{-18}
compact	512	0 / 1	0.0281 s	1.134 s	40.44x	5.32×10^{-16}
boundary crossing	128	18 / 0	0.158 s	0.710 s	4.50x	4.32×10^{-17}
oxygen-heavy	2,047	0 / 27	0.647 s	8.918 s	13.78x	5.10×10^{-16}

All integrals agreed with the requested measure and no positivity repair was required.

Full-trajectory field evidence

Using all 1,500 trajectory frames, the exact 10^{-8} cutoff, and the canonical effective-CIC grid:

Channel	Grid	Source nodes	Direct/FFT tiles	Hybrid realization	LD7 field time	Realization speedup
Na	540^3	36,280	10 / 105	2.088 s	41.934 s	20.08x
Si	1038^3	54,680	14 / 94	2.015 s	54.244 s	26.91x
Al	1037^3	57,471	12 / 107	2.146 s	56.022 s	26.11x

The three fields recovered exactly 24 atoms. Peak process RSS was approximately 0.95–0.98 GiB on the validation runtime. A small number of finite-support-boundary FFT nodes required exact direct repair: 10 for Na, 1 for Si, and 0 for Al.

A full production oxygen realization was not completed in this S3 validation session because repeated construction of the pre-existing S1 oxygen atlas showed unstable wall time under the shared execution environment. The separate oxygen-heavy S3 fixture passed against S2, and the prior exact S1 oxygen atlas remains validated. LD8-S4 must rerun the complete four-species production benchmark before authorizing default migration.

Stage boundary

LD8-S3 is complete when the hybrid executor and selector pass the focused numerical and resource tests. It does **not** make the hybrid path the production default.

LD8-S4 remains responsible for:

- full Na/Si/Al/O production benchmarking;
- selector recalibration and cold/warm-cache evidence;
- integration into scene planning and normal field dispatch;
- block extrema and exact multi-HDR query reuse;
- final wall-time, memory, fallback, and positivity-repair gates;
- authorization or rejection of production-default migration.

External algorithmic source

The overlap-add block-convolution organization follows the standard treatment in:

A. V. Oppenheim, R. W. Schaffer, and J. R. Buck, *Discrete-Time Signal Processing*, 2nd ed., Prentice Hall, 1999.

The periodic packed support, triclinic-metric discrete kernel, tile selector, exact target lookup, and supported-node repair are mdstats-specific designs.